EXPRESS.JS FRAMEWORK

Theory Assignment:

Introduction:

What is express.js?

Express.js is a fast, minimal and flexible web framework for node.js

It helps developers build web servers and APIs easily

Advantages of express.js over the built-in http module

Express.js : easy to use, less code, built-in routing, middleware support, easy handling of requests/responses, faster development

http module: more complex, more code, manual routing, no direct middleware support, manual handling required, slower development

main advantages of express.js

1) Simple routing

Express makes route creation easy:

Ex: app.get("/about", (req, res)=>{

Res.send("about page); });

2) Middleware support

Express supports middleware function for:

Loggin, authentication, validation, error handling

Ex: app.use((req, res, next) =>{

Console.log(req,method);

Nex();});

3) Cleaner code
   Express reduce boilerplate code and improves readability.

4) Easy handling of requests & responses
   Express provides:
   Req.params
   Req.query
   Req.body
   This makes data handling easier

5) Rest API development
   Express is widely used for creating restful apis quickly

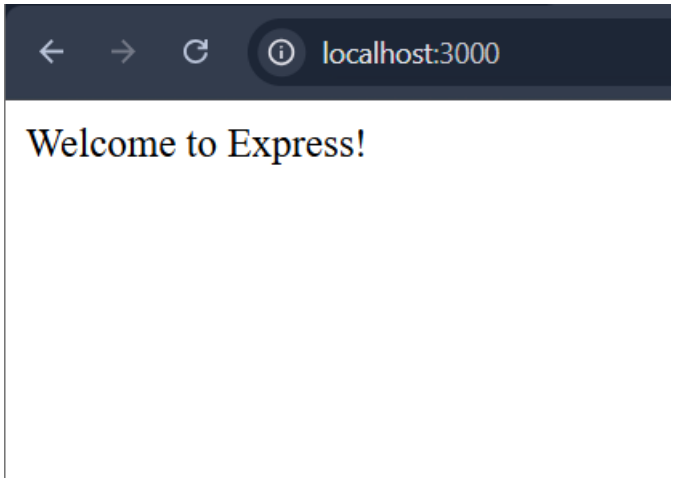
Practical assignment:

1) Setting up a basic express server
   Create project folder
   Mkdir express-app
   Cd express-app
   Npm init -y
   Npm install express

   Server.js

```
const express = require("express");

const app = express();

const PORT = 3000;

// Root Route
app.get("/", (req, res) => {
```

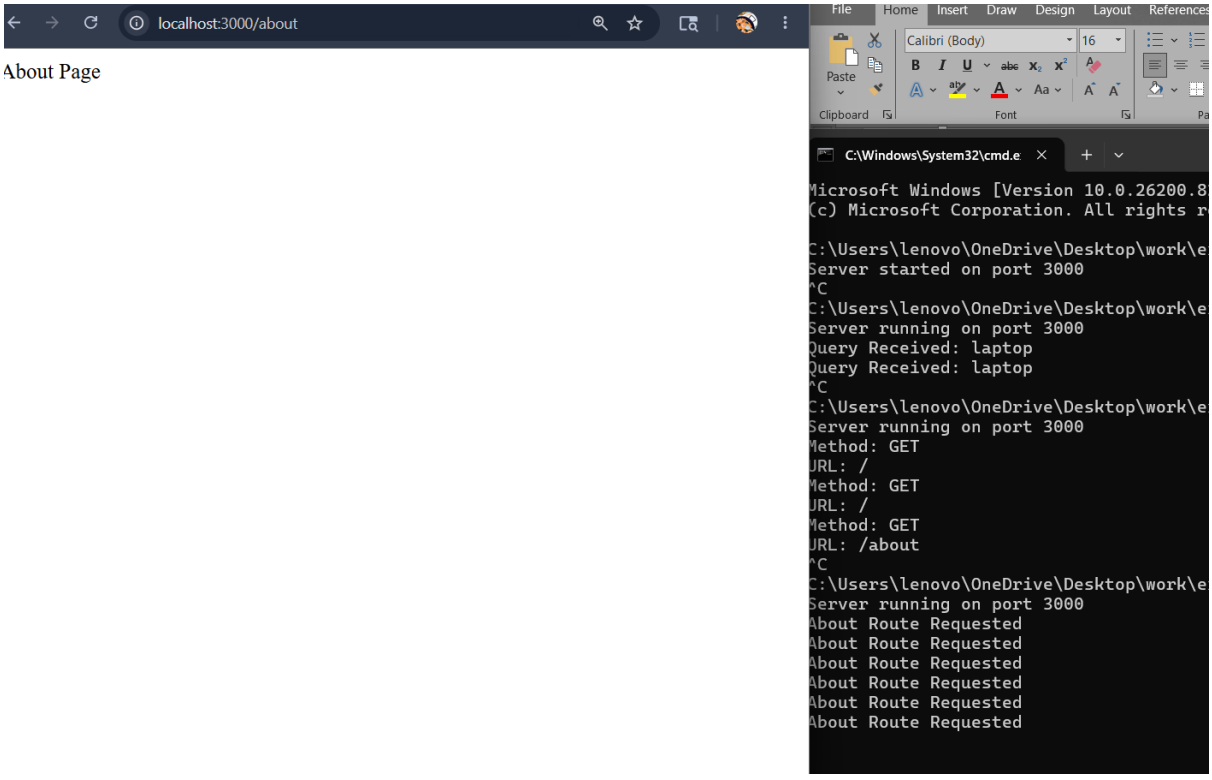
```
    res.send("Welcome to Express!");
});

// Server Start
app.listen(PORT, () => {
    console.log(`Server started on port ${PORT}`);
});
```



2) Creating routes: multiple routes

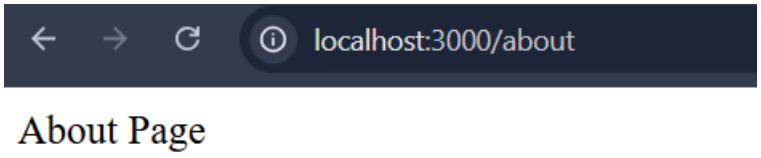
```
const express = require("express");

const app = express();

const PORT = 3000;

// Home Route
app.get("/", (req, res) => {
    console.log("Home Route Requested");
    res.send("Home Page");
});
```

```
// About Route
app.get("/about", (req, res) => {
   console.log("About Route Requested");
   res.send("About Page");
});

// Contact Route
app.get("/contact", (req, res) => {
   console.log("Contact Route Requested");
   res.send("Contact Page");
});

// Server Start
app.listen(PORT, () => {
   console.log(`Server running on port ${PORT}`);
});
```



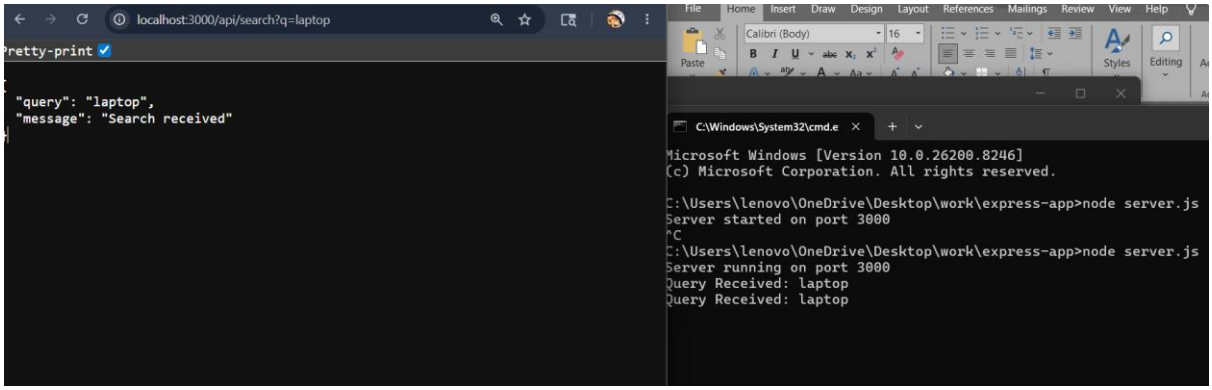
3) Using express middleware

```
const express = require("express");

const app = express();

const PORT = 3000;

// Custom Middleware
app.use((req, res, next) => {
   console.log(`Method: ${req.method}`);
   console.log(`URL: ${req.url}`);

   next(); // Important
});

// Routes
app.get("/", (req, res) => {
   res.send("Home Page");
});

app.get("/about", (req, res) => {
   res.send("About Page");
});

// Server Start
app.listen(PORT, () => {
   console.log(`Server running on port ${PORT}`);
});
```



4) Handling query parameters:

```
const express = require("express");

const app = express();

const PORT = 3000;

// Search Route
app.get("/api/search", (req, res) => {

   const searchTerm = req.query.q;

   console.log("Query Received:", searchTerm);

   res.json({
      query: searchTerm,
      message: "Search received"
   });
});

// Server Start
app.listen(PORT, () => {
   console.log(`Server running on port ${PORT}`);
});
```



5) Get, post, delete routes example:

```
const express = require("express");

const app = express();

const PORT = 3000;

// Middleware
app.use(express.json());

// GET Route
app.get("/", (req, res) => {
   res.send("GET Request");
});

// POST Route
app.post("/user", (req, res) => {
```

```
    console.log(req.body);

    res.json({
        message: "POST Request Successful",
        data: req.body
    });
});

// DELETE Route
app.delete("/delete/:id", (req, res) => {

    const userId = req.params.id;

    res.json({
        message: "Delete Request Successful",
        deletedId: userId
    });
});

// Server Start
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

Important express methods

Method

App.get() => fatch data

App.post()=> insert data

App.put()=> update data

App.delete() => delete data

App.use() => middleware

res.send() => send response

Res.json()=> send json

Handling form data with post requests

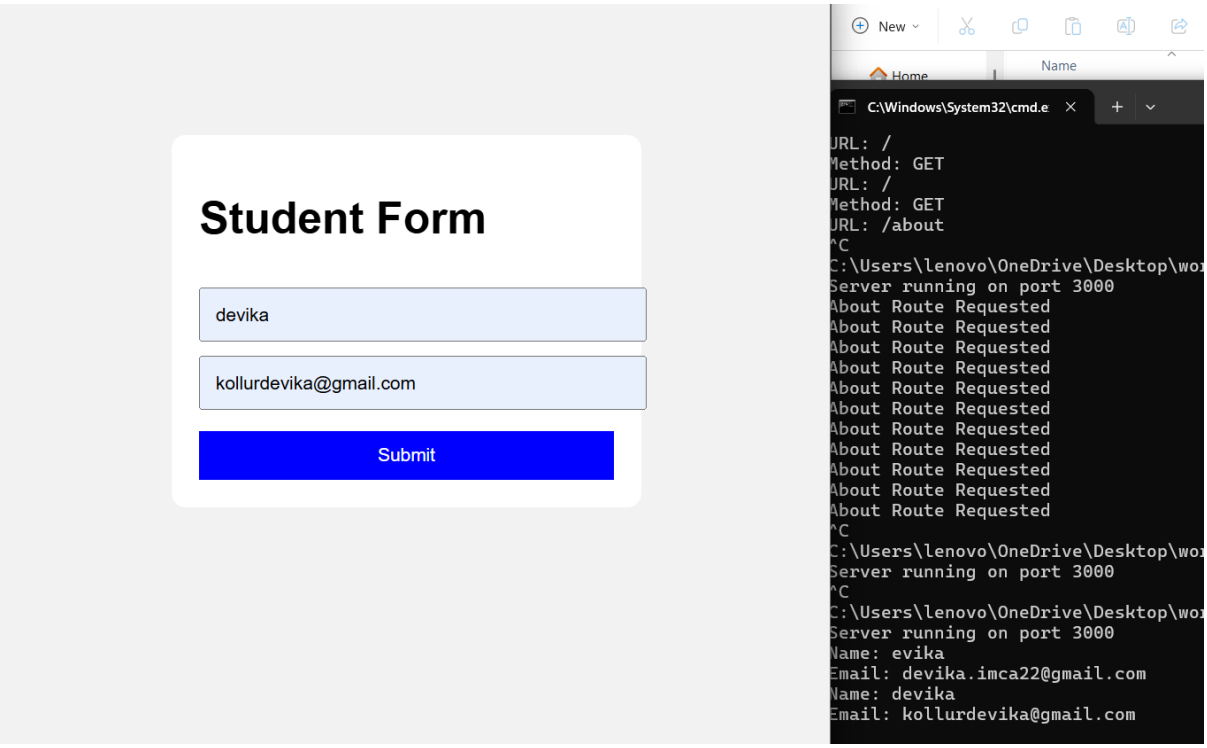
Server.js

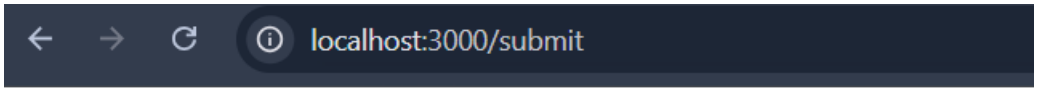
```
const express = require("express");
const path = require("path");

const app = express();

const PORT = 3000;

// Middleware for form data
app.use(express.urlencoded({ extended: true }));

// Serve Static Files
app.use(express.static("public"));
```

```
// Root Route
app.get("/", (req, res) => {
   res.sendFile(path.join(__dirname, "public", "index.html"));
});

// POST Route for Form Submission
app.post("/submit", (req, res) => {

   const { name, email } = req.body;

   console.log("Name:", name);
   console.log("Email:", email);

   res.send(`
      <h1>Form Submitted Successfully</h1>
      <h2>Name: ${name}</h2>
      <h2>Email: ${email}</h2>
   `);
});

// Start Server
app.listen(PORT, () => {
   console.log(`Server running on port ${PORT}`);
```

});

Output:


Student Form
devika
kollurdevika@gmail.com
Submit
New
Name
Home
C:\Windows\System32\cmd.e
URL: /
Method: GET
URL: /
Method: GET
URL: /about
^C
C:\Users\lenovo\OneDrive\Desktop\wo
Server running on port 3000
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
About Route Requested
^C
C:\Users\lenovo\OneDrive\Desktop\wo
Server running on port 3000
^C
C:\Users\lenovo\OneDrive\Desktop\wo
Server running on port 3000
Name: evika
Email: devika.imca22@gmail.com
Name: devika
Email: kollurdevika@gmail.com



Form Submitted Successfully

Name: devika

Email: kollurdevika@gmail.com

Understanding important part:

1) Express.urlencoded()
   App.use(express.urlencoded({ extended: true}));
   Used to read form data from req.body

2) Serving static files

   App.use(express.static("public"));

   This servers:

   Html, css, javascript, images from the public folder

3) form post method
   <from action="/submit" method="post">

Action=”submit”=> sends data to /submit
Method=”post”=> send data securely

4) access from data
const {name, email } = req.body;

implementing error handling middlewre

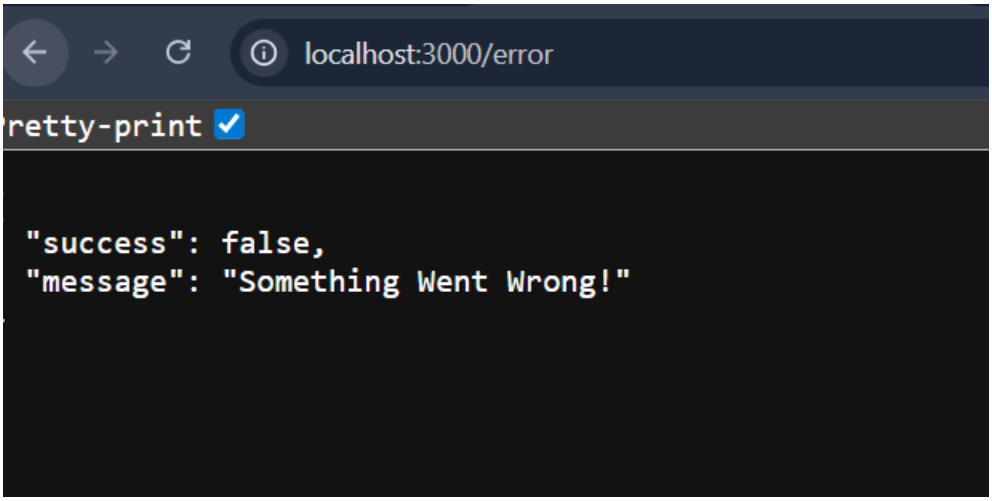
what is error handling middleware?
In express.js error handling middleware is used to:
Catch application errors
Prevent server crashes
Send proper error responses
(err, req, res, next)

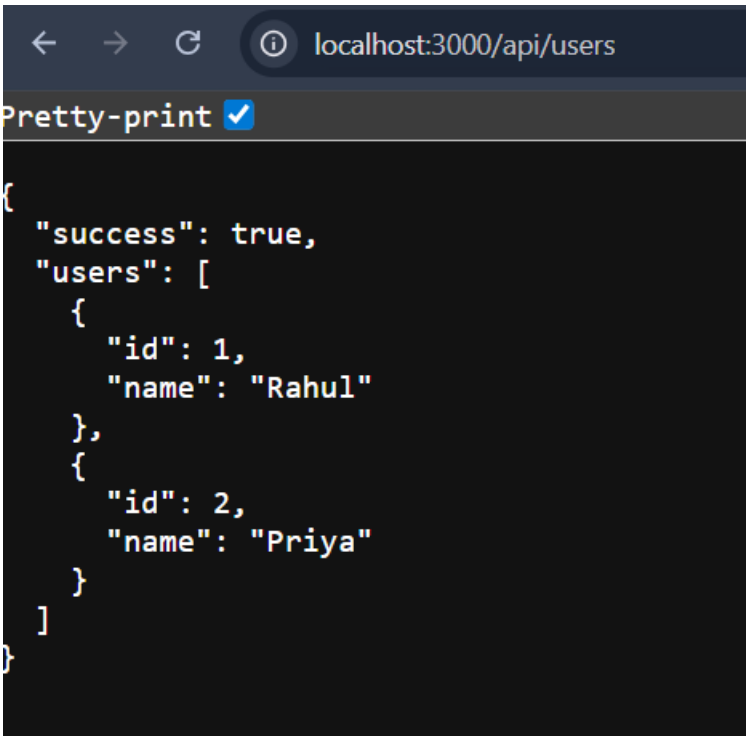
Server.js

```
const express = require("express");

const app = express();

const userRoutes = require("./routes/users");

const PORT = 3000;

// Middleware
app.use(express.json());


// Home Route
```

```
app.get("/", (req, res) => {
   res.send("Home Page");
});


// Route to Simulate Error
app.get("/error", (req, res, next) => {

   const error = new Error("Something Went Wrong!");

   next(error);
});


// User Routes
app.use("/api/users", userRoutes);



// Error Handling Middleware
app.use((err, req, res, next) => {

   console.log("Error Middleware Called");

   res.status(500).json({
      success: false,
      message: err.message
   });
});


// Server Start
app.listen(PORT, () => {
```

```
    console.log(`Server running on port ${PORT}`);
});
```





1) express router:
   const router = express.router();
   used to create moduler routes.
2) Import router

Const userRoutes = require(“./routes/users”);
Imports routes into main app

3) Use router
   App.use(“/api/users”, useRoutes);
   Base route: /api/users

4) Error middleware
   App.use((err, req, res, next)=> {
   Catches all application errors

5) Next(error)
   Next(error);
   Passes error to error middleware

   Why use router?

   Clean code
   Moduler structure
   Easy maintenance
   Better project organization